

Техническое задание - ТЗ

База знаний для ChatGPT (Codex) через GitHub

Версия: 2.0

Дата: *2025-12-03*

Срок выполнения: 1 месяц

Исполнитель: 1 разработчик

Стоимость: 150 000 рублей

---

1. Цель проекта

Создать минимально рабочую систему (MVP), которая:

1. принимает от пользователя файлы формата **DOC/DOCX**;
2. автоматически конвертирует их в **Markdown**;
3. размещает полученный Markdown в **GitHub-репозитории**;
4. делает эти файлы доступными ChatGPT/Codex через GitHub integration.

Система должна работать полностью автоматически:

```
Загрузка → Конвертация → Публикация → Доступ ChatGPT
```

---

2. Назначение системы

Создаётся **Knowledge Base (База знаний)** на GitHub, из которой ChatGPT (через Codex) сможет:

- читать документы,
- использовать полученные данные в диалогах.

---

3. Участники и компоненты

Источники:

- Пользователь (загрузка файлов)

Компоненты системы:

- Backend API
- MinIO (входящее хранилище файлов)
- Argo Workflows (pipeline обработки)
- Pandoc (конвертация DOC → MD)
- GitHub API (публикация MD)

Потребитель данных:

- ChatGPT / Codex (через GitHub integration)

---

4. Общий процесс (Flow)

```
Пользователь → Backend API → MinIO → Argo Workflow → Pandoc → Markdown → GitHub → ChatGPT
```

---

5. Рабочий процесс подробно (MVP Flow)

5.1 Шаг 1. Загрузка документа пользователем

Frontend (минимальный):

- форма для загрузки 1 файла DOC/DOCX

- отправка на Backend через `multipart/form-data`

На стороне Backend:

1. Принять файл.
2. Обработка ошибок
3. Логи и базовая мониторинг-система.
4. Валидация максимального размера файлов.
5. Проверить формат (*.doc*/.docx).
6. Сгенерировать путь:

```
uploads/<user_id>/<yyyy-mm-dd>/<uuid>.docx
```

1. Загрузить файл в MinIO.
2. Стартовать Argo Workflow, передав параметры:
   - `input_file`
   - `output_file`
   - `git_path`

5.2 Шаг 2. Argo Workflow

Workflow состоит из **4 шагов**:

Step 1 — загрузка оригинала из MinIO

В контейнере с `minio/mc`:

```
mc cp minio/bucket/<input_file> /tmp/input.docx
```

Step 2 — конвертация DOC → Markdown

В контейнере с `pandoc`:

```
pandoc /tmp/input.docx -f docx -t markdown -o /tmp/output.md
```

Требование:

Без пост-обработки → получить чистый Markdown.

Step 3 — загрузка результата в MinIO

```
mc cp /tmp/output.md minio/bucket/<output_file>
```

Step 4 — загрузка Markdown в GitHub

Через GitHub API:

```
PUT /repos/<org>/<repo>/contents/<git_path>
```

Контент — base64.

---

6. Структура GitHub-репозитория

```
/docs
   /knowledge
      /<user_id>/
         <yyyy-mm-dd>-<uuid>.md
```

Пример:

```
docs/knowledge/user123/2025-01-01-abc123.md
```

---

7. Функциональные требования MVP

Backend:

- Принимать DOC/DOCX файлы
- Валидировать формат
- Сохранять оригинал в MinIO
- Генерировать корректные пути
- Запускать Argo Workflow с параметрами
- Возвращать пользователю статус “принято в обработку”

Argo Workflow:

- Выполнять 4 шага:
  1. Download (MinIO → Pod)
  2. Convert (Pandoc)
  3. Upload (Pod → MinIO)
  4. Publish to GitHub API
- Обработка только happy-flow (без retries, без backoff)

GitHub Knowledge Base:

- Принимать Markdown через GitHub API
- Размещать по пути, указанному Backend

---

8. Требования к интеграции с ChatGPT (Codex)

Система должна обеспечивать:

- наличие файлов в GitHub в Markdown-формате,
- доступ к этим файлам через GitHub OAuth,
- корректный парсинг Markdown (заголовки, списки, таблицы),

- возможность задавать ChatGPT вопросы по документам.

---

9. Финальный результат (Definition of Done)

После загрузки DOC/DOCX:

✔️ файл сохраняется в MinIO

✔️ запускается Argo Workflow

✔️ Workflow скачивает файл

✔️ конвертирует его в Markdown через Pandoc

✔️ загружает Markdown в MinIO

✔️ публикует Markdown в GitHub по заданному пути

✔️ файл становится доступным через GitHub для ChatGPT Codex

Проверка:

- Открываем файл на GitHub → Markdown отображается корректно
- ChatGPT в Codex может прочитать документ

---

10. Сдача и приёмка MVP

Исполнитель должен предоставить:

1. Репозиторий с Backend API (Go/Node — по согласованию)
2. Argo Workflow YAML
3. Dockerfile'ы и манифесты для всех контейнеров
4. Инструкцию по развёртыванию:
    - настройки MinIO
    - настройки GitHub API
    - запуск Workflow

    - переменные окружения

5. Полный рабочий end-to-end flow

- Регистрация домена (если требуется публичный доступ).
- Настройка сервера/ВМ: ОС, firewall, SSH, базовая безопасность.
- Установка и конфигурация всех сервисов:
    - Backend API (Go/Node/Python)
    - MinIO (хранилище файлов)
    - Argo Workflows
    - Pandoc
    - Docker / Kubernetes (если используется)
- Настройка среды окружения и переменных
- Настройка логирования и базового мониторинга
- Тестирование end-to-end flow на реальном сервере.